

CSED312 Project 2

Design Report

팀 오우노

20230355 정지성

20230345 이성재

목차

1. 현재의 pintos implementation에 대한 분석
 - 1.1. Process execution 과정
 - 1.2. System call 과정
 - 1.3. File system
2. Design Proposal
 - 2.1. Process termination messages
 - 2.2. Argument passing
 - 2.3. System call
 - 2.4. Denying writes to executables

1. 현재의 pintos implementation에 대한 분석

1.1. Process execution 과정

1.1.1. Loading 과정: 최초 실행부터 idle thread 생성까지

Pintos를 맨 처음 실행시켰을 때, PC BIOS가 로더 프로그램 `threads/loader.S`을 메모리에 load한다.

`loader.S`의 역할은 디스크에서 커널을 찾고 이를 메모리에 load한 뒤, 커널의 entry point 주소로 jump하는 역할을 한다. 이렇게 해서 실행되는 함수가 `threads/start.S`의 `start()`이며, 이 파일은 기기의 메모리 크기 확인, page table initialization 등의 low-level kernel initialization을 진행한다.

이를 완료하면 `threads/init.c` 파일의 `main()` 함수로 점프하여, 이 파일에서 high-level kernel initialization이 시작된다. `bss_init()`, `thread_init()`, `console_init()`, ... 등등 이후 커널의 실행에 필요한 정말 많은 것에 대한 초기화가 진행되고, 이 과정에서 idle thread가 생성된다. 마지막으로 `run_actions()`에서 실행 시 받은 command line input에 따라서 action들을 실행한다.

`threads/init.c`의 `main()` 함수에서 Project 1과 비교해서 달라진 점이 있다면, `#ifdef USERPROG`와 `#endif`로 둘러싸인 부분에서 user program 구현을 위한 initialization이 추가로 실행된다는 것이다. 이에는 `exception_init()`, `syscall_init()`, `tss_init()`, 그리고 `gdt_init()`이 있다.

- `exception_init()`과 `syscall_init()`은 interrupt vector table 중 synchronous interrupt에 대한 vector table entry를 initializing하는 코드가 담겨 있다. 특히 `syscall_init()`에서 우리가 구현하는 syscall handler 함수에 대한 initialization을 한다.
- `gdt_init()`는 x86 아키텍처에 사용되는 segment를 설정한다. `tss_init()`는 user stack과 kernel stack을 교환하는 데에 사용하는 TSS를 initialize한다.

또 `process_wait (process_execute (task));`을 통해 첫 process를 실행하는 것을 확인할 수 있다.

마지막으로, `threads/thread.h`에서, `struct thread`에 `uint32_t *pagedir`이 추가된 것을 확인할 수 있다. 이는 user virtual memory의 매핑 테이블으로, 이 table이 있으면 현재 user program의 virtual address를 physical address로 번역할 수 있다.

1.1.2. ELF Binary가 load되는 과정

ELF Binary가 user process로 load되는 과정을 알아보기 위해서는 `load()` 함수를 뜯어보아야 한다.

아래에서 함수를 구간별로 나누어 설명하겠다.

```
bool
load (const char *file_name, void (**eip) (void), void **esp)
{
    struct thread *t = thread_current ();
    struct Elf32_Ehdr ehdr;
    struct file *file = NULL;
    off_t file_ofs;
    bool success = false;
    int i;

    /* Allocate and activate page directory. */
    t->pagedir = pagedir_create ();
    if (t->pagedir == NULL)
        goto done;
    process_activate ();

    /* Open executable file. */
    file = filesys_open (file_name);
    if (file == NULL)
    {
        printf ("load: %s: open failed\n", file_name);
        goto done;
    }
}
```

먼저 thread의 page directory를 allocate하고 activate해 준다. 여기서 page directory를 activate한다는 것은 그 page directory를 이용해 virtual memory의 address를 해석하겠다는 것이다. 그 후, 실행하고자 하는 파일의 이름을 받아서 `filesys_open()` 함수를 통해 함수를 연다.

```
/* Read and verify executable header. */
if (file_read (file, &ehdr, sizeof ehdr) != sizeof ehdr
    || memcmp (ehdr.e_ident, "\177ELF\1\1\1", 7)
    || ehdr.e_type != 2
    || ehdr.e_machine != 3
    || ehdr.e_version != 1
    || ehdr.e_phentsize != sizeof (struct Elf32_Phdr)
    || ehdr.e_phnum > 1024)
{
    printf ("load: %s: error loading executable\n", file_name);
    goto done;
}
```

여기서는 ELF의 executable header을 읽고, ELF가 load 가능한 파일인지 검증하는 과정이다. 예를 들어, 우리 architecture와 맞지 않거나 type이 executable이 아닌 ELF들이라면 에러를 남기고 종료한다.

```

/* Read program headers. */
file_ofs = ehdr.e_phoff;
for (i = 0; i < ehdr.e_phnum; i++)
{
    struct Elf32_Phdr phdr;

    if (file_ofs < 0 || file_ofs > file_length (file))
        goto done;
    file_seek (file, file_ofs);

    if (file_read (file, &phdr, sizeof phdr) != sizeof phdr)
        goto done;
    file_ofs += sizeof phdr;
    switch (phdr.p_type)
    {
        case PT_NULL:
        case PT_NOTE:
        case PT_PHDR:
        case PT_STACK:
        default:
            /* Ignore this segment. */
            break;
        case PT_DYNAMIC:
        case PT_INTERP:
        case PT_SHLIB:
            goto done;
        case PT_LOAD:
            if (validate_segment (&phdr, file))
            {
                bool writable = (phdr.p_flags & PF_W) != 0;
                uint32_t file_page = phdr.p_offset & ~PGMASK;
                uint32_t mem_page = phdr.p_vaddr & ~PGMASK;
                uint32_t page_offset = phdr.p_vaddr & PGMASK;
                uint32_t read_bytes, zero_bytes;
                if (phdr.p_filesz > 0)
                {
                    /* Normal segment.
                     Read initial part from disk and zero the rest. */
                    read_bytes = page_offset + phdr.p_filesz;
                    zero_bytes = (ROUND_UP (page_offset + phdr.p_memsz, PGSIZE)
                                - read_bytes);
                }
                else
                {
                    /* Entirely zero.
                     Don't read anything from disk. */
                    read_bytes = 0;
                    zero_bytes = ROUND_UP (page_offset + phdr.p_memsz, PGSIZE);
                }
                if (!load_segment (file, file_page, (void *) mem_page,
                                read_bytes, zero_bytes, writable))
                    goto done;
            }
            else
                goto done;
            break;
    }
}
}

```

여기서는 ELF의 program header를 분석하고 그에 맞게 행동하는 부분이다. 주목할 부분은 `phdr.p_type`가 `PT_LOAD`인 경우인데, 그 경우에는 segment가 메모리에 load되어야 한다는 뜻으로, `load_segment()` 함수를 이용해 load해 준다.

```

/* Set up stack. */
if (!setup_stack (esp))
    goto done;

/* Start address. */
*eip = (void (*)(void)) ehdr.e_entry;

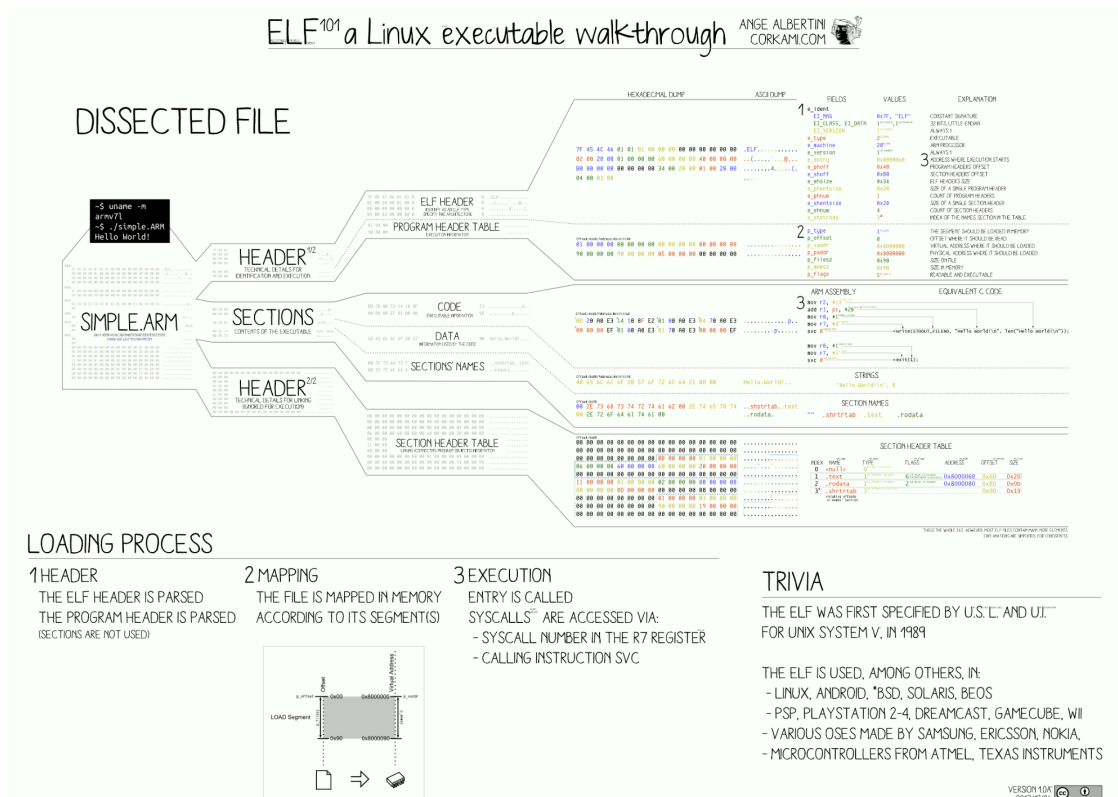
success = true;

done:
/* We arrive here whether the load is successful or not. */
file_close (file);
return success;
}

```

이후 stack을 만들어 그 pointer를 esp에 넘겨 주고, 코드의 start address는 eip에 넘겨 준다. 모든 과정을 성공했으면 true를, 어느 한 군데라도 문제가 발생했다면 false를 반환하고 마무리한다.

참고로, 아래는 executable header와 program header에 대해 설명해 주는 그림이다. 여기서의 각각의 field가 Pintos의 `struct Elf32_Ehdr`와 `struct Elf32_Phdr`에 정의되어 있음을 확인할 수 있다.



1.1.3. process.c에 정의된 함수들

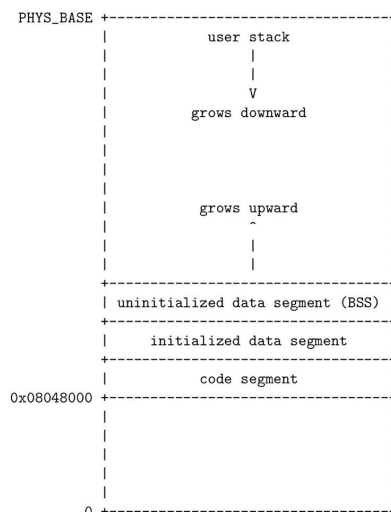
`process_execute()`, `start_process()`, `process_wait()`, `process_exit()`, `process_activate()`가 있다.

- `process_execute()`는 실행할 `file_name`을 받아, `start_process()` 함수에 `file_name`을 arg로 넣은 것을 실행하는 thread를 만들어 주는 함수이다.
- `start_process()`는 앞서 설명한 `load()` 함수를 통해 page directory로 virtual memory를 initialize하고, user program을 이 virtual memory에 load한다. 이후, `intr_frame`을 만든 뒤 `jmp intr_exit`을 하는 방식으로 user mode으로 들어간다. 이게 가능한 이유는, 여기서의 `intr_exit` 명령어 이전까지는 kernel의 context에서 코드를 읽었기 때문이다.
- `process_wait()`는 아직 구현되지 않은 함수이다.
- `process_exit()`는 user process의 page directory를 부수는 역할이다. Thread이 `thread_exit()`를 할 때 부른다.
- `process_activate()`는 current thread의 `struct thread`에 저장된 `pagedir`을 activate해 주는 과정이다. 여기서 page directory를 activate한다는 것은 그 page directory를 이용해 virtual memory의 address를 해석하겠다는 것이다. 이 함수는 두 군데에서 불린다. 처음으로는 앞서 봤듯이 `load()` 함수에서 page directory를 처음 만들 때이다. 두 번째로는 context switching을 할 때마다 `thread_schedule_tail()`에서 불린다. 이 덕분에 context switching을 할 때 user process의 virtual memory가 교환될 수 있다.

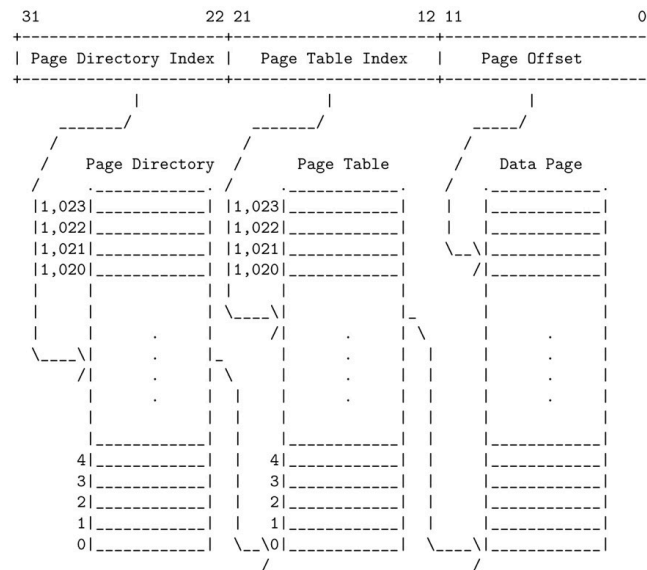
1.1.4. Virtual Memory

이 단락에서는 user program의 virtual memory 구조를 알아본다.

- 우선, user virtual memory와 kernel virtual memory로 나뉜다. User virtual memory는 다음과 같이 생겼다. 여기서 `PHYS_BASE`는 `0xc0000000` (3GB)로 기본설정 되어 있다.



- Kernel virtual memory는 **PHYS_BASE**에서부터 시작된다. 여기는 Pintos의 physical address space를 일대일로 복사해 온 것과 마찬가지로 address가 매핑되어 있다. 예를 들어, virtual memory의 **PHYS_BASE + 0x2000**이 가리키는 physical address는 **0x2000**이다.
- User virtual memory는 page directory에 따라서 physical memory로 매핑된다. CSED211이나 CSED311에서 배운 것처럼, page table은 다음과 같이 주소를 적절히 쪼갠 뒤 두 번의 indexing을 통해 physical address를 저장한다.



- Stack의 setup은 virtual address에서 PHYS_BASE 바로 아래 가상페이지를 physical address로 매핑해야 한다. 이 때 physical memory에서 페이지 하나를 받아 온 후, **install_page()** 함수를 이용해 page directory와 page table의 적절한 엔트리에 이 page address를 설정한다.
- code의 loading은 header에 적혀있는 곳부터 virtual memory에 세팅을 한다. 이것도 비슷하게 physical page를 필요한 개수만큼 할당하여 그 곳에 저장한 후, 남은 공간은 0으로 초기화한다. 이후 **install_page()** 함수를 이용해 virtual address와의 연결을 짓는다.
- Context switching이 일어날 때 virtual memory가 교환되는 방식은 1.1.4.에서 설명하였다.

1.2. System call 과정

먼저 system call이 무엇을 의미하는지 정의를 분명히 해야 한다. System call은 trap의 일종으로, user program에 의해 의도적으로 일어나는 software interrupt이다. User program이 커널 영역에서 수행하고 싶은 동작이 있을 경우 system call을 일으켜 주도권을 커널에 넘길 수 있다. 커널 영역에서 수행할 일들이 종료되면 다시 user 영역으로 돌아와 나머지 코드를 실행시킬 수 있다.

User program은 `int 0x30` 과 같은 명령어를 이용해 system call을 일으킬 수 있다. 해당 interrupt가 발생하면 `intr-stubs.S` 에서 정의된 `intr_stub()` 이 실행된다. 해당 어셈블리 코드에서 매크로로 정의되어 있기 때문에 이 경우 정확히는 `intr30_stub()` 함수가 실행될 것이다. 이 함수의 주소는 IDT, 즉 Interrupt Descriptor Table에 저장되어 있다. CPU가 자동으로 `intr30_stub()` 함수를 실행시키면 user stack의 정보들과 함께 error code 등이 커널 스택에 저장된다. 이후 `intr_entry`의 `call intr_handler` 부분이 실행되면 `src/threads/interrupt.c`의 `intr_handler()` 함수가 실행된다.

`intr_handler()` 함수는 `intr_stub`에서 만든 `intr_frame`을 이용해서 구체적인 handler 함수를 호출한다. 이때 어떤 handler를 사용할지는 `src/userprog/exceptions.c` 의 `exception_init()` 함수에서 초기화를 해놓았다. 만약 handler가 지정되지 않았다면 `unexpected_interrupt()`가 해당 interrupt를 처리하게 된다.

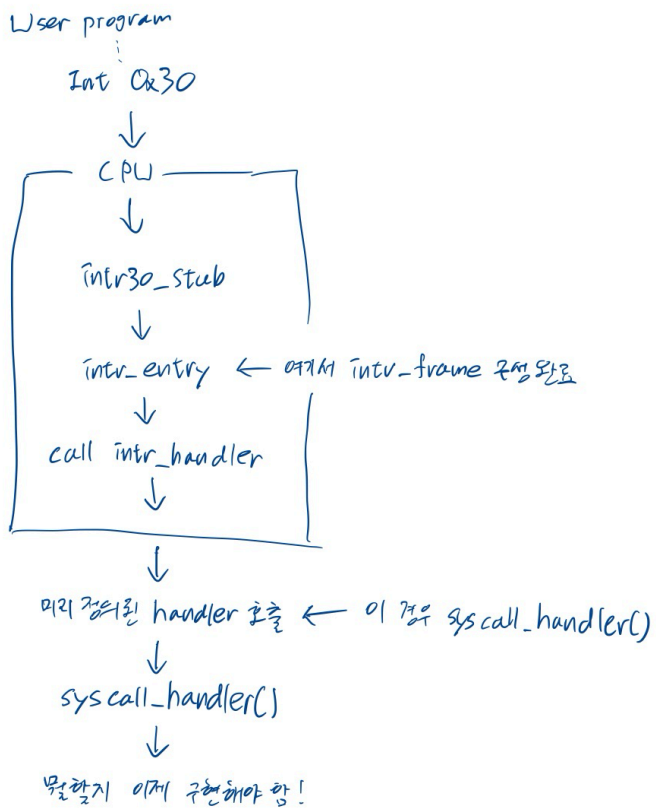
0x30을 `vec_no`로 갖는 interrupt, 즉 system call에 대한 handler는 `syscall_handler()` 함수로 초기화되어 있다.

```
void
syscall_init (void)
{
    intr_register_int (0x30, 3, INTR_ON, syscall_handler, "syscall");
}
```

위의 코드는 `src/userprog/syscall.c` 의 일부로, syscall 이라는 이름을 가진 interrupt를 정의한 모습을 볼 수 있다.

이제 `syscall_handler()` 함수가 실행됐다. 하지만 아직 구현은 제대로 되어 있지 않고, 이를 구현하는 것이 이번 과제의 목표 중 하나이다. 현재는 `system call!` 이라는 문장을 프린트하고 `thread_exit()`을 호출하는 것이 전부다. 즉 현재 코드는 syscall을 처리하지 않고, syscall이 일어나면 즉시 해당 스레드를 종료시키고 있다. 이를 각 syscall에 대해 적절한 처리를 하는 코드로 수정해야 한다.

위의 과정들을 다이어그램으로 나타내면 대략 다음과 같다.



1.3. File system

Project 2의 주 목표는 file system과 거리가 있음에도 이 부분의 코드를 분석해야 하는 이유는 두 가지가 있다. 첫째로는 user program이 파일 시스템에서부터 load된다는 것이다. 두 번째는 우리가 구현해야 하는 system call이 file system과 관련있는 경우가 많다는 것이다.

1.3.1. Simulated disks

Pintos에서 파일을 사용하기 위해서는, Pintos 코드를 위한 simulated disk를 만들고 사용할 줄 알아야 한다. Hard drive를 emulate하는 파일을 만드는 것이라고 생각하면 되는데, Pintos는 이것을 통해서만 파일 시스템을 생성하고 관리할 수 있다. Simulated disk는 `pintos-mkdist filesys.dsk` `-filesys-size=2` 명령어를 통해 만들 수 있다. 또 `pintos -f -q`를 사용해서 simulated disk를 초기화할 수 있다.

컴퓨터에 있던 ELF 파일을 simulated disk에 옮기기 위해서는 `pintos -p (file location) -a (name in simulated disk) -- -q` 명령어를 사용할 수 있다. 또, 이를 실행하기 위해서는 `pintos -q run '(file to run, with arguments)'`를 사용하면 된다.

1.3.2. 파일 시스템 및 파일의 전반적 관리 (filesys/filesys.c)

이 단락에서는 파일 시스템을 중추적으로 관리하는 코드인 `filesys.c`에 정의된 함수들의 기능을 살펴 본다.

- `filesys_init() / filesys_done()`: 파일 시스템을 현재 부팅 상태에서 활성화하거나 끝내는 역할을 하는 함수들이다.
- `filesys_create()`: 위의 init에 의해 사용중인 파일 시스템에 새로운 파일 하나를 생성하는 함수이다. 파일이름과 초기 파일 크기를 input으로 받는다. 성공 여부를 bool로 반환한다.
- `filesys_open()`: 파일 시스템에 저장되어 있는 파일 중 하나를 실행하는 함수이다. 파일 이름을 input으로 받는다. **성공 시 struct file 포인터를 반환한다. 연 파일을 수정할 때에는 여기서 받은 struct file에서 작업해야 한다.**
- `filesys_remove()`: 파일 시스템에 저장되어 있는 파일 중 하나를 삭제하는 함수이다. 파일 이름을 input으로 받는다. 성공 여부를 bool로 반환한다.

1.3.3. 파일을 읽고 수정하는 방법 (filesys/file.c)

이 단락에서는 `filesys/file.c` 코드를 읽어 보며 파일의 읽기와 쓰기를 수행하는 함수들을 알아본다. 먼저 `open`을 실행한 file마다 배정되는 `struct file`의 구조를 살펴 보자.

```
/* An open file. */
struct file
{
    struct inode *inode;      /* File's inode. */
    off_t pos;               /* Current position. */
    bool deny_write;         /* Has file_deny_write() been called? */
};
```

열린 file마다 배정되는 struct이다. `inode`에는 disk에서의 file의 location 등 file의 여러 정보가 담겨 있다. `pos`는 파일에서 커서의 위치를 나타내는 숫자이다. `deny_write`가 true라면 이 파일에는 write를 하면 안 된다.

→ 기본적으로, `struct file`은 파일의 `inode`에 커서와 쓰기방지 기능 두 개를 추가한 형태라고 보아주면 된다.

다음으로, `filesys/file.c`에 정의된 함수들을 기능별로 묶어 역할을 분석하겠다.

1.3.3.1. 파일을 여닫는 함수

- `file_open()` / `file_reopen()`: `struct inode`를 input으로 받아, `struct file`을 만들어 반환해 준다. `file_reopen()`의 경우 `struct inode` 대신 이미 열려 있던 file의 `struct file`을 input으로 받는다.
- `file_close()`: `struct file`로 받은 파일을 닫는다. File과 엮인 `inode`도 같이 닫아준다.
- `file_get_inode()`: `struct file`을 받아 그 file의 `struct inode`를 반환한다.

1.3.3.2. 파일을 읽고 쓰는 함수

- `file_read()`: `struct file`과 buffer pointer, 그리고 size (길이)를 받는다. 현재 커서 위치부터 시작해서, size만큼의 byte를 buffer에 저장한다. 실제로 읽은 길이를 반환한다.
- `file_read_at()`: `file_read()`와 비슷한데, file offset의 위치도 같이 받는다. 이렇게 하면 현재 커서 위치부터가 아닌 input으로 받은 file offset부터 읽는다.
- `file_write()`: `struct file`과 buffer pointer, 그리고 size (길이)를 받는다. buffer에 있는 내용 중 size만큼을 현재 커서 위치부터 시작해 저장한다. 실제로 쓴 길이를 반환한다.
- `file_write_at()`: `file_write()`와 비슷한데, file offset의 위치도 같이 받는다. 이렇게 하면 현재 커서 위치부터가 아닌 input으로 받은 file offset부터 읽는다.

1.3.3.3. 파일 쓰기를 관리하는 함수

- `file_deny_write(): struct file`을 input으로 받아, file의 `deny_write`를 켜다. 이 과정에서 file과 연관된 inode에게도 deny write를 해 준다.
- `file_allow_write(): struct file`을 input으로 받아, file의 `deny_write`를 끈다. 이 과정에서 file과 연관된 inode에게도 allow write를 해 준다.

1.3.3.4. 파일의 포지션 관련 함수

- `file_seek(): struct file`와 offset을 받아, file의 커서의 위치를 파일 시작으로부터 offset만큼 더한 곳으로 설정한다.
- `file_tell(): struct file`을 받아 그 file의 커서 위치를 반환한다.
- `file_length(): struct file`을 받아, 파일의 byte 기준 길이를 반환한다.

1.3.4. Inode의 구조 (filesys/inode.c)

이 단락에서는 `filesys/inode.c`에 정의된 `struct inode`들을 구경해 보며 파일 불러오기, 읽기/쓰기 동작이 더 low한 레벨에서 대략적으로 어떻게 이루어지는지 본다. `filesys/inode.c`에 정의된 함수들은 우리가 직접 부를 함수들은 아닐 가능성이 매우 높아, 우리의 project 2를 구현하는 데에 있어 도움이 되지 않을 것 같아 생략했다.

우선 두 종류의 inode가 있음을 인지해야 한다. 첫번째는 디스크에 파일과 함께 상주하는 `inode_disk`인데, 여기에는 `inode_disk`에 해당하는 파일의 디스크 위치와, 파일의 byte기준 크기를 저장한다. 즉, disk에 저장되어 있는 파일 하나마다 `inode_disk` 하나가 달려 있다고 생각하면 된다. `inode_create()` 함수를 이용해 만들 수 있다.

```
/* On-disk inode.
   Must be exactly BLOCK_SECTOR_SIZE bytes long. */
struct inode_disk
{
    block_sector_t start;           /* First data sector. */
    off_t length;                  /* File size in bytes. */
    unsigned magic;                /* Magic number. */
    uint32_t unused[125];          /* Not used. */
};
```

두 번째는 파일을 읽을 때 메모리에 load하는 inode이다. 기본적으로는 `inode_disk`에 추가적인 플래그를 달아 놓은 형태라고 생각하면 된다. Pintos kernel이 파일을 읽고 싶을 때에는 `inode_open()`을 통해 파일에 해당하는 `inode_disk`를 찾고 이를 기반으로 `inode`를 만들어 메모리에 저장하는 방식으로 파일 읽기/쓰기를 지원한다.

```
/* In-memory inode. */
struct inode
{
    struct list_elem elem;          /* Element in inode list. */
    block_sector_t sector;          /* Sector number of disk location. */
    int open_cnt;                  /* Number of openers. */
    bool removed;                  /* True if deleted, false otherwise. */
    int deny_write_cnt;             /* 0: writes ok, >0: deny writes. */
    struct inode_disk data;         /* Inode content. */
};
```

2. Design Proposal

2.1 Process termination messages

Process를 terminate할 때, process name과 exit code를 출력하는 기능을 추가하면 된다. Process를 돌리지 않는 kernel thread에 대해서는 출력하면 안 된다. 또, halt system call에 의한 종료는 출력하지 않는다.

이 기능을 구현하기 위해서는 메시지를 어디에 출력할 지만 잘 정하면 될 것으로 보인다. `process_exit()` 함수가 적절해 보인다.

2.2. Argument passing

Command line argument을 적절히 parsing해서 stack에 올리는 것까지 완료하면 되는 task이다. 다음 state of the stack을 참고해, 아래의 과정을 거치면 task를 완료할 수 있을 것으로 예상된다. 이 로직을 위해 따로 추가해야 하는 data structure는 없어 보인다.

1. Stack에 command line argument 통채로 push하기
2. Stack pointer을 한 byte씩 읽으면서, whitespace를 만나면 null byte로 바꾸고, whitespace의 다음 주소를 기억해 두기 (argument의 개수도 argc에 keeping track하기)
3. Word align이 되도록 null byte 1~4개 push하기
4. 2에서 기억한 주소들을 하나하나 push하기
5. 마지막으로, argv의 베이스 주소와 argc값 push하고, return address 0 push하기

e.g., State of the stack at the beginning of the user program with the command “/bin/ls -l foo bar”

```
bfffffc0 00 00 00 00 | .....|
bfffffd0 04 00 00 00 d8 ff ff bf-ed ff ff bf f5 ff ff bf |.....|
bfffffe0 f8 ff ff bf fc ff ff bf-00 00 00 00 2f 62 69 |...../bi|
bffffff0 6e 2f 6c 73 00 2d 6c 00-66 6f 6f 00 62 61 72 00 |n/l.s.-l.foo.bar.|
```

Address	Name	Data	Type
0xbffffc	<u>argv</u> [3][...]	bar\0	char[4]
0xbffff8	<u>argv</u> [2][...]	foo\0	char[4]
0xbffff5	<u>argv</u> [1][...]	-l\0	char[3]
0xbffffed	<u>argv</u> [0][...]	/bin/l\0	char[8]
0xbffffec	word-align	0	uint8_t
0xbffffe8	<u>argv</u> [4]	0	char *
0xbffffe4	<u>argv</u> [3]	0xbffffc	char *
0xbffffe0	<u>argv</u> [2]	0xbffff8	char *
0xbffffdc	<u>argv</u> [1]	0xbffff5	char *
0xbffffd8	<u>argv</u> [0]	0xbffffed	char *
0xbffffd4	<u>argv</u>	0xbffffed8	char **
0xbffffd0	<u>argc</u>	4	int
0xbffffcc	return address	0	void (*) ()

2.3 System call

크게 두 단계로 나누어 구현을 할 수 있을 것 같다.

2.3.1. System call infrastructure 설계

먼저 구현해야 할 것은 system call number를 읽고, 이에 따라 정확한 handler를 대응시키는 기능이다.

`intr_frame`의 `esp`를 이용하는 switch문을 만들어 이를 구현할 수 있다. `error_code`가 가질 수 있는 값들과 그 의미는 `src/lib/syscall-nr.h`에 enumerator로 정의되어 있다.

2.3.2. 각 system call 설계

구현해야 하는 system calls를 두 종류로 나눠보자면, 다른 user process를 컨트롤 할 수 있게 하는 system call과 file에 접근하는 system call로 나눌 수 있다. 이를 project2 description pdf에서는 각각 User Process Manipulation, 그리고 File Manipulation으로 정의했고 해당 보고서에서도 이를 따를 예정이다.

2.3.2.1. User Process Manipulation

> `halt()` : `devices/shutdown.h`의 `shutdown_power_off()` 함수를 호출한다.

> `exit()` : User program을 종료시키고 커널에게 status를 넘겨야 한다. Thread 구조체에 exit status를 관리하는 변수를 추가하고, 해당 변수에 매개변수로 들어온 status 값을 넣는 것으로 구현할 수 있다. 스레드를 종료시키는 것은 `thread_exit()` 함수를 호출해서 수행할 수 있다.

> `exec()` : `cmd_line`에 들어온 executable을 실행시키는 system call이다. 이 함수 내에선 `process_execute()`을 호출해야 하고, `process_execute()`도 수정해야 한다. `process_execute()`은 내부적으로 `thread_create()`을 통해 자식 스레드를 만들고, 해당 스레드에서 executable을 로딩하므로 이를 완전하게 구현하려면 `process_execute()`을 수정해야 한다.

```
tid_t
process_execute (const char *file_name)
{
    char *fn_copy;
    tid_t tid;

    /* Make a copy of FILE_NAME.
       Otherwise there's a race between the caller and load(). */
    fn_copy = pallocc_get_page (0);
    if (fn_copy == NULL)
        return TID_ERROR;
    strcpy (fn_copy, file_name, PGSIZE);

    /* Create a new thread to execute FILE_NAME. */
    tid = thread_create (file_name, PRI_DEFAULT, start_process, fn_copy);
    if (tid == TID_ERROR)
        pallocc_free_page (fn_copy);
    return tid;
}
```

위의 코드는 기존에 구현되어 있는 `src/userprog/process.c`의 `process_execute()` 함수로, 다음과 같은 문제가 있다.

- i. 로딩이 실패해도 `tid`가 반환됨
- ii. 부모 프로세스가 반환되기 전에 자식 프로세스가 실행-종료될 수 있음(자식 추적 전에 자식이 사라질 수 있음)
- iii. `start_process`에서 오류가 발생해도 부모가 감지할 방법이 없음

위의 세 문제를 해결하도록 `process_execute()` 함수를 수정해야 한다.

> `wait()` : 자식 프로세스가 종료 되길 기다리도록 구현해야 한다. 이는 `src/userprog/process.c`의 `process_wait()` 함수를 구현하고 호출하는 방식으로 작성할 수 있다. `process_wait()`은 자식이 종료될 때까지 기다리고, 자식의 `exit_status`를 반환해야 한다. 이는 자식이 종료될 때 semaphore을 1증가시키고 부모는 `sema_down()`을 하도록 해 구현할 수 있다. 물론 고려해야 될 edge case들이 있다. 자식이 `exit()`을 호출하지 않고 커널에 의해 종료된 경우 `process_wait()`은 -1을 반환해야 한다. 주어진 `pid`가 현재 프로세스의 자식이 아닌 경우 `wait()`이 -1을 반환하고 종료되도록 해야한다. 또한 이미 `wait()` 중인 자식 프로세스에 대해 다시 `wait()`을 한 경우에도 -1을 반환하고 종료되도록 해야한다. 먼저 `process_wait()`을 구현하고, `wait()`이 뒤의 두 edge cases를 처리할 수 있도록 구현할 계획이다.

2.3.2.2. User Process Manipulation

Data structure for file descriptor: thread 구조체에 file descriptor를 관리하는 table을 만들어야 한다.

```
struct thread {
    struct file *fd_table[128];
    int next_fd;
    struct file *executable;
    // ...
};
```

또한 핀토스의 파일 시스템은 atomic하지 않기 때문에 파일에 접근할 때 사용할 lock을 만들어야 한다.

`file_open()` 등의 함수를 사용할 때 전후에 lock을 acquire하고 release하는 과정이 필요하다.

> `create()` : 먼저 파일의 이름과 크기의 유효성을 검사한다. 하지만 이는 이미 `filesys_create()`에 구현이 되어 있다. 정확히는 `filesys_create()`이 success값을 구하기 위해 호출하는 함수들 안에 구현이 되어 있다. 따라서 lock을 얻고 `filesys_create()`를 호출한 다음 lock을 release하는 것으로 충분하다. 반환값으로는 `filesys_create()`의 반환값을 넘겨주면 된다.

> `remove()` : `create()`에서와 비슷하게, `filesys_remove()`를 호출하고 그 반환값을 넘겨주는 것으로 충분하다. 물론 호출 전후에 lock을 얻고 해제하는 과정은 필요하다.

> `open()` : 먼저 `file_open()`을 이용해 파일을 열고, file descriptor table에 추가한다. `fd_table`의 비어있는 index에 파일을 추가하고(`next_fd`), `next_fd`를 1증가시킨 다음 해당 fd를 반환한다. 위의 과정 중에 실패한 과정이 있다면 -1을 반환한다.

> `filesize()` : 매개변수로 주어진 fd에 해당하는 파일을 `fd_table`에서 찾고, 해당 파일을 매개변수로 `file_length()`를 호출한다. 그 반환값을 넘겨주면 된다.

> `read()` : fd가 0, 즉 stdin을 의미하는 경우 size 횡수만큼 반복문을 돌며 `input_getc()`를 호출해 키보드 입력을 받는다. 이후 입력받은 횡수를 반환하는데, 이 값은 i+1로 계산할 수 있다. 입력 마지막에는 null문자가 추가되기 때문이다. fd가 1(stdout을 의미)이면 즉시 -1을 반환한다. fd가 1보다 크다면, `fd_table`에서 파일을 가져오고, `file_read()`를 호출한다.

> `write()` : fd가 0이면 즉시 -1을 반환한다. fd가 1이면 `putbuf()`를 호출한다. fd가 1보다 크면 `fd_table`에서 파일을 가져오고 `file_write()`를 호출한다.

> `seek()` : `fd_table`에서 파일을 가져오고 `file_seek()`를 호출한다.

> `tell()` : `fd_table`에서 파일을 가져오고 `file_tell()`를 호출한다.

> `close()` : `fd_table`에서 파일을 가져온다. 이때 파일이 `null`이면 오류를 발생시키는 것이 아니라 무시하도록 작성해야 한다. `file_close()`를 호출하고, `fd_table[fd]`를 `null`로 바꾼다. 또한 `fd`가 0, 1인 경우 파일을 닫을 수 없으므로 `null`인 경우와 비슷하게 무시하도록 작성해야 한다.

2.4 Denying writes to executables

Process가 실행하고 있는 코드의 파일들에 대해 deny write를 켜는 기능을 구현해야 한다. 우선 이 기능을 구현하는 이유는, process가 수정중인 코드를 실행할 때 많은 예상불가능한 문제가 발생할 수 있기 때문이다.

우선 process의 tcb인 `struct thread` data structure에 현재 실행중인 코드의 `struct file` 정보를 저장하는 것이 좋아 보인다. 여기서 저장한 `struct file`은 process가 실행되기 시작할 때부터 끝나기 직전까지 작성을 금지해야 한다. 그런데 file을 닫으면 write가 enable될 것이다. 그래서, process가 처음 실행될 때 `file_deny_write()`를 적절히 부르고, `file_allow_write()`는 부르는 대신 file을 닫는 것으로 대체해도 좋아 보인다.

이제 결정할 것은 `file_deny_write()`를 어디에서 부를 것이냐에 관한 문제이다. 이는 ELF binary를 user process로 로드하는 `load()` 함수 (`userprog/process.c`에 위치함)에 배치하는 것이 적절해 보인다.